

ME5411 Robot Vision and AI Report

Group 9

Deng Yanwei-A0285336X

Lu Zixuan-A0285204J

Yang Fuyuan-A0285179M



November 21, 2023

Contents

1	Introduction	2
1.1	Problem Overview	2
1.2	Objectives	2
1.3	Report Structure	3
2	Questions and Solutions	3
2.1	Question 1	3
2.2	Question 2	4
2.3	Question 3	10
2.4	Question 4	11
2.5	Question 5	12
2.6	Question 6	14
2.7	Question 7	18
2.7.1	Data Preparation and Division	18
2.7.2	Convolutional Neural Networks (CNN)	19
2.7.3	Multilayer Perceptrons (MLP)	20
2.7.4	Results	21
2.7.5	Comparison	23
2.8	Question 8	24
2.8.1	Pre-processing of Data	24
2.8.2	Hyperparameter Tuning	30
3	Summary	35

1 Introduction

The ME5411 Computer Project in Robot Vision and AI presents an opportunity to delve into the realm of image processing and machine learning using MATLAB. The task at hand involves the analysis and classification of characters in a microchip label image (Figure 1). This report aims to introduce the problem, outline the algorithm and flowchart, showcase the image processing stages, and discuss the chosen methods and their outcomes.

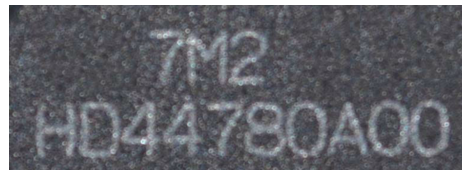


Figure 1: image of a label on a microchip

1.1 Problem Overview

The project begins with the display of the original image, followed by the implementation and application of various image smoothing techniques using averaging and rotating masks. Subsequent steps involve sub-image extraction, binary image creation, character outlining, and segmentation. The dataset provided is then divided into training and validation sets for the development of a Convolutional Neural Network (CNN) and a non-CNN-based classification system.

1.2 Objectives

In a broader context, the project strives to provide participants with fundamental skills and profound insights in the realms of image processing and machine learning. It seeks to cultivate a deeper comprehension of how these skills apply to cutting-edge technologies. The key tasks encompass the following:

1. Display the original microchip label image.
2. Apply averaging and rotating masks for image smoothing and compare results.
3. Extract a sub-image containing the middle line HD44780A00.
4. Create a binary image using thresholding.
5. Determine character outlines in the image.
6. Segment and label different characters.

7. Design a CNN and a non-CNN-based classification system for character recognition, compare results, and analyze the effectiveness and efficiency of the approaches.
8. Experiment with preprocessing techniques and hyperparameter tuning, assessing the sensitivity of the chosen approach to these changes.

1.3 Report Structure

This report will provide a detailed account of each task, supported by algorithm descriptions, flowcharts, and screen captures at every stage of image processing. The subsequent discussion and conclusion will delve into the rationale behind method selection, the investigations conducted, and the lessons learned throughout the project.

2 Questions and Solutions

2.1 Question 1

Display the original image on the screen

In this initial step of the image processing project, our objective was to showcase the original microchip label image. The MATLAB code begins by loading the image from the specified path (*img_input\charact2.bmp*). Subsequently, the image is converted to grayscale using the *im2gray* function, simplifying subsequent processing steps.

Partial code of main_task.m

```
% Q1: Display the original image on the screen
fprintf('Showing original image...\n');
originalImage = imread('img_input\charact2.bmp');
myImage = im2gray(originalImage);
imwrite(myImage, 'img_output\myImage.png');
figure;
imshow(myImage);
title('Original Image');
```

The code provides a graphical display of the grayscale image in a new MATLAB figure window, accompanied by the title *Original Image*. After the visual representation of the original image in MATLAB the grayscale version is saved as a *.PNG* file for reference and further analysis (*img_output\myImage.png*).

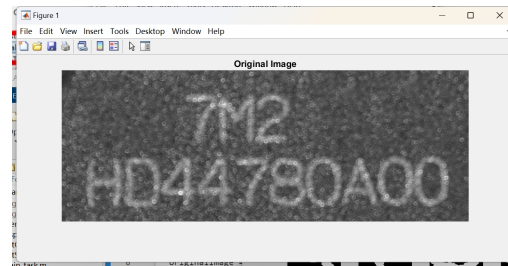


Figure 2: The program execution results for Question 1

2.2 Question 2

Implement and apply a 3x3 averaging mask and a 3x3 rotating mask on the image. Experiment with masks of different sizes. Compare and comment on the results of the respective image smoothing methods.

Image masks are fundamental tools in image processing that involve convolution operations. A mask, also known as a kernel or filter, is a small matrix applied to each pixel in an image. The values in the mask define the weights for neighboring pixels, and the convolution operation calculates a weighted sum, producing a smoothed or filtered result. Averaging masks, for instance, assign equal weights to all pixels, while rotating masks introduce directional emphasis.

In the algorithm, two essential functions, `rotateFilter` and `averageFilter`, are established to implement the image smoothing process.

- a) **Averaging Mask:** This function utilizes an averaging mask, distributing equal weights to neighboring pixels. The filter size again plays a crucial role in determining the scale of the smoothing operation. This MATLAB function, `'averageFilter'`, is designed to apply an averaging filter to smooth the input grayscale image, `'myImage'`. The filter used is a square matrix of ones, normalized by the square of its size (`'filterSize'`). The function iterates through each pixel of the image, calculating the average value within a local neighborhood defined by the filter size. The resulting smoothed pixel values are stored in the output image, `'smoothedImage'`.

The algorithm initializes the output image as a matrix of zeros with the same size as the input image. It then defines the averaging filter kernel by creating a matrix of ones and normalizing it by the square of the filter size. The function uses nested loops to traverse each pixel in the input image, extracts the local neighborhood centered around the pixel, and computes the average value by applying the filter kernel.

The calculated average is assigned to the corresponding pixel in the output image, resulting in the final smoothed image. This process effectively reduces high-frequency variations in pixel intensity, achieving a smoothing effect on the input image. The `neighborhoodRadius` variable ensures that the filter is applied within the bounds of the image, preventing edge artifacts.

averageFilter.m

```
function smoothedImage = averageFilter(myImage, filterSize)
    % averageFilter applies an averaging filter to smooth the
    ↪ input image.
    % The filter is a square matrix of ones normalized by the
    ↪ square of its size.
    % Get the size of the image
    [rows, cols] = size(myImage);
    % Define the averaging filter kernel
    filter = ones(filterSize) / (filterSize^2);
    % Initialize a result image of the same size as the
    ↪ original image
    smoothedImage = zeros(size(myImage));
    % Calculate the half-size of the neighborhood
    neighborhoodRadius = floor(filterSize / 2);
    % Perform average filtering
    for i = neighborhoodRadius + 1 : rows -
    ↪ neighborhoodRadius
        for j = neighborhoodRadius + 1 : cols -
    ↪ neighborhoodRadius
            % Extract the neighborhood
            neighborhood = double(myImage(i -
    ↪ neighborhoodRadius : i + neighborhoodRadius, j -
    ↪ neighborhoodRadius : j + neighborhoodRadius));

            % Apply the averaging filter kernel
            smoothedPixel = sum(sum(neighborhood .* filter));

            % Update the pixel value in the result image
            smoothedImage(i, j) = smoothedPixel;
        end
    end
end
```

- b) **Rotating Mask:** This function applies a rotating mask to the image, emphasizing directional features. The choice of filter size determines the extent of the smoothing effect. The creation approach for the `rotateFilter` function is similar to that of the `averageFilter`, both relying on the common convolution operation in image processing for achieving image smoothing. Differing from `averageFilter`, the convolution kernel of `rotateFilter` is a rotating matrix designed to emphasize directional features in the image.

Similarly, a nested loop is employed to traverse each pixel in the image, extracting local neighborhoods centered around each pixel, and then calculating the average using the convolution kernel. The computed average is assigned to the corresponding pixel in the output image, achieving the desired smoothing effect on the input image.

rotateFilter.m

```
function smoothedImage = rotateFilter(myImage, filterSize)
    % rotateFilter applies a rotating filter to smooth the
    ↪ input image.
    % The filter is created by convolving a ones matrix with
    ↪ itself,
    % and then normalizing the result by dividing by the
    ↪ fourth power of filterSize.

    % Get the size of the image
    [rows, cols] = size(myImage);

    % Initialize the result image
    smoothedImage = zeros(rows, cols);

    % Define the filter weights
    filter0 = conv2(ones(filterSize), ones(filterSize));
    filter = filter0 / power(filterSize, 4);

    % Perform rotating filtering
    for i = filterSize : rows - filterSize + 1
        for j = filterSize : cols - filterSize + 1
            % Extract the neighborhood
            neighborhood = double(myImage(i - filterSize + 1
    ↪ : i + filterSize - 1, j - filterSize + 1 : j + filterSize
    ↪ - 1));

            % Apply the convolution kernel
            smoothedPixel = sum(sum(neighborhood .* filter));

            % Update the pixel value in the result image
            smoothedImage(i, j) = smoothedPixel;
        end
    end
end
```

Following the image smoothing code segment, the *averageFilter* and *rotateFilter* functions are subsequently invoked in the main program, *main_task.m*, to apply smoothing to the desired image. This MATLAB code systematically applies averaging and rotating masks of different sizes for image smoothing to the input image *myImage*. The resulting smoothed images are visually compared

and saved in the `img_output\MaskedImages` directory.

Partial code of main_task.m

```
fprintf('Applying different masks for image smoothing...\n');
filterSizes = [3, 5]; % Different filter sizes
methods = {'Averaging', 'Rotating'}; % Two methods: Averaging and
↪ Rotating
figure;
index = 1;
for i = 1:length(filterSizes)
    filterSize = filterSizes(i);
    for j = 1:length(methods)
        method = methods{j};
        if strcmp(method, 'Averaging')
            smoothedImage = uint8(averageFilter(myImage,
↪ filterSize));
        elseif strcmp(method, 'Rotating')
            smoothedImage = uint8(rotateFilter(myImage,
↪ filterSize));
        else
            error('Invalid smoothing method');
        end
        % Display the smoothed image
        subplot(2, 2, index);
        index = index + 1;
        imshow(smoothedImage);
        title(sprintf('%dx%d %s mask result', filterSize,
↪ filterSize, method));
        % Save the smoothed image to the specified output
↪ directory
        outputFilename = sprintf('%dx%d %s mask result.png',
↪ filterSize, filterSize, method);
        outputPath = fullfile('img_output\MaskedImages',
↪ outputFilename);
        imwrite(smoothedImage, outputPath);
    end
end
```

Afterwards, to contrast the differences in various image smoothing methods, histograms were plotted. Histograms are graphical representations that showcase the distribution of pixel intensities in an image. They provide valuable insights into how different smoothing techniques impact the overall distribution of intensity values, aiding in the comparative analysis of their effects.

Partial code of main_task.m

```
% Loop through each image file and generate histograms
for i = 1:numel(imageFiles)
    % Construct the full file path
    imagePath = fullfile(folderPath, imageFiles(i).name);

    % Generate a title using the file name
    [~, titleText, ~] = fileparts(imageFiles(i).name);
    titleText = ['Histogram of ', titleText];

    % Plot histograms on the same figure
    subplot(2, ceil(numel(imageFiles)/2), i);
    plotImageHistogram(imagePath, titleText);
end
```

In this context, the *plotImageHistogram* function is used to generate histograms for images and is defined as follows:

plotImageHistogram.m

```
function plotImageHistogram(imagePath, titleText)
    image = imread(imagePath);
    % Calculate the histogram
    [counts, binLocations] = imhist(image);
    % Plot the histogram
    bar(binLocations, counts);
    % Add title and labels
    title(titleText);
    xlabel('Gray Level');
    ylabel('Pixel Count');
end
```

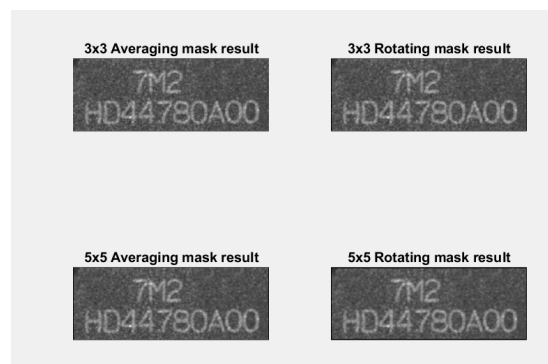


Figure 3: Applying different masks for image smoothing

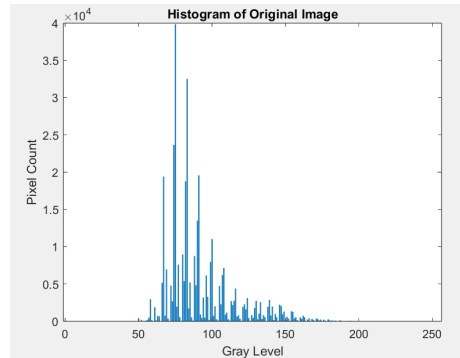


Figure 4: The histogram of the original image

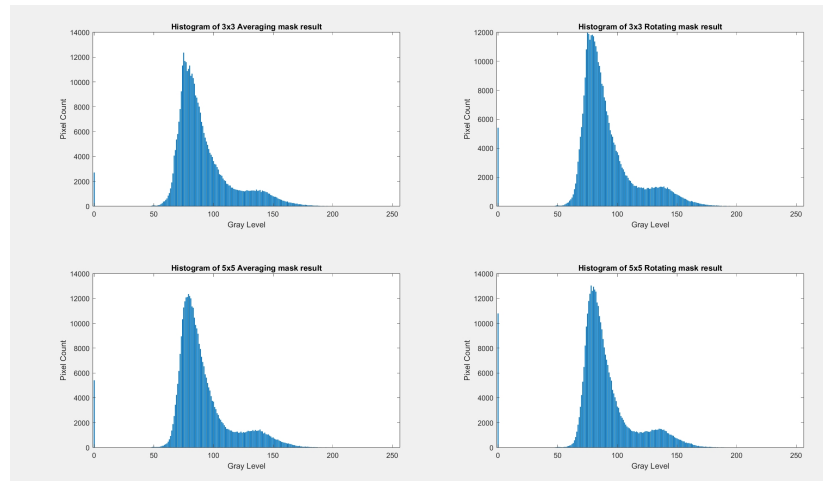


Figure 5: Histograms of different image smoothing methods

We compared the 3x3 average mask, 3x3 rotation mask, 5x5 average mask, and 5x5 rotation mask, and plotted histograms for each. The following observations can be made:

- **Effect of Masks:** Both masks can increase the pixel count, making the distribution of the image histogram more even (smoothing effect).
- **Size of Masks:** The 3x3 average mask and 3x3 rotation mask have similar effects on image smoothing, while the 5x5 average mask and 5x5 rotation mask introduce a larger averaging area, resulting in a more pronounced blur effect.
- **Comparison between Average and Rotation Masks:** In the comparison, it was found that the average mask tends to produce an overall

smoothing effect, while the rotation mask tends to preserve the edge features of the image. This suggests that choosing different types of masks can achieve adjustments and optimizations for different aspects of the image.

2.3 Question 3

Create an image which is a sub-image of the original image comprising the middle line – HD44780A00.

Based on the earlier presentation of the original image, it is evident that the second line of text to be extracted is located in the lower portion of the image. We can use a rectangular shape to crop the image. The code is as follows:

getSecondLine.m

```
function secondLineImage = getSecondLine(myImage, outputFilename)
    % getSecondLine extracts a sub-image corresponding to the
    ↪ second line from the input image.
    % It also highlights the region of interest and saves the
    ↪ original and extracted images.
    if nargin < 2
        outputFilename = 'Image';
    end
    % Define the region of interest for the second line
    secondLineRegion = [1, 190, 990-1, 367-190];
    % Create a figure without displaying it
    h = figure('Visible', 'on');
    % h = figure('Visible', 'off');
    % Show the original image
    imshow(myImage);
    % Highlight the region on the original image
    hold on;
    rectangle('Position', secondLineRegion, 'EdgeColor', 'r',
    ↪ 'LineWidth', 3);
    % Save the original image with the highlighted rectangle
    saveas(h, fullfile('img_output', ['highlightedSecondLine',
    ↪ outputFilename, '.png']));
    % Extract the second line region
    secondLineImage = imcrop(myImage, secondLineRegion);
    % Save the extracted second line image
    imwrite(secondLineImage, fullfile('img_output', ['secondLine',
    ↪ outputFilename, '.png']));
    % Close the figure
    close(h);
end
```

Partial code of main_task.m

```
% Q3: Create an image which is a sub-image of the original image
↳ comprising the line:HD44780A00.
fprintf('Creating a sub-image comprising the line:
↳ HD44780A00...\n');
secondLineImage = getSecondLine(myImage);
```

Here, to ensure a reasonable cropping area, we visualize the area to be cropped. Adjust the size of the cropping area to a suitable range, perform the cropping operation, and finally, save the cropped image.



Figure 6: Create a sub-image

2.4 Question 4

Create a binary image from Step 3 using thresholding.

Thresholding is a fundamental image processing technique where pixel intensity values are used to separate an image into distinct regions. In this case, the *imbinarize* function is employed to create a binary image (*binaryImage.png*) from the *secondLineImage.png* using a specified threshold value of 0.47 (the optimal values obtained after multiple experiments).

main_task.m

```
...
fprintf('Creating a binary image using thresholding...\n');
thresholdValue = 0.47;
binaryImage = imbinarize(secondLineImage, thresholdValue);

subplot(2, 3, 3);
imshow(binaryImage);
title('Binary Image');
...
```

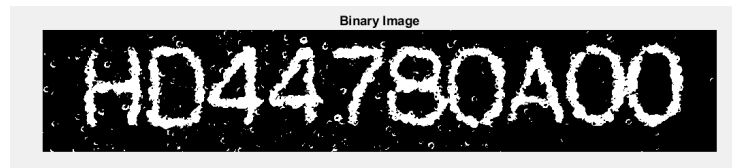


Figure 7: Binary image created by thresholding

Pixels with intensity values above this threshold are assigned one value (typically white), while pixels below the threshold are assigned another value (typically black). This process effectively converts the grayscale image into a binary representation, highlighting regions of interest based on their intensity levels.

2.5 Question 5

Determine the outline(s) of characters in the image

The *getOutline* function processes a binary image by applying morphological operations and Canny edge detection. It includes erosion and dilation to reduce noise, a cleaning step to remove small areas, and edge detection to highlight region boundaries. This function performs the following steps:

1. **Morphological Operations:**
 - a) Erosion (imerode): This operation involves "shrinking" the foreground regions in the binary image by removing pixels from their boundaries. It helps to smooth the edges and reduce small details or noise.
 - b) Dilation (imdilate): Conversely, dilation "expands" the foreground regions by adding pixels to their boundaries. It helps to enhance and connect regions, making them more prominent.
2. **Cleaning Operation (bwareaopen):**

After dilation, the function uses *bwareaopen* to eliminate small connected components (areas) in the binary image that have a pixel count less than 100. This step helps to remove tiny artifacts and ensure a cleaner representation.
3. **Edge Detection (edge with Canny):**

The cleaned binary image undergoes Canny edge detection using the *edge* function. The Canny edge detector identifies rapid intensity changes, highlighting edges in the image. This results in an image where prominent boundaries between regions are emphasized.
4. **Customizable Filenames and Image Output:**

The function allows users to specify custom filenames for intermediate results or uses default names. Intermediate images, including the eroded

image, dilated image, cleaned image, and edge-detected image, are saved for further analysis or documentation.

getOutline.m

```
function edgeImage = getOutline(binaryImage, varargin)
    % getOutline applies morphological operations and edge
    ↪ detection to a binary image.
    se1 = strel('disk', 2);
    se2 = strel('disk', 4);
    % Default filenames
    defaultFilenames = {'erodedImage', 'dilatedImage',
    ↪ 'cleanedImage', 'edgeImage'};
    % Parse user-provided filenames or use defaults
    if nargin > 1
        userFilenames = varargin;
    else
        userFilenames = defaultFilenames;
    end
    % Perform erosion operation
    erodedImage = imerode(binaryImage, se1);
    imwrite(erodedImage, ['img_output/', userFilenames{1},
    ↪ '.png']);
    % Perform dilation operation
    dilatedImage = imdilate(erodedImage, se2);
    imwrite(dilatedImage, ['img_output/', userFilenames{2},
    ↪ '.png']);
    % Use bwareaopen to hard cut off remaining small areas with
    ↪ pixel count < 100
    cleanedImage = bwareaopen(dilatedImage, 100);
    imwrite(cleanedImage, ['img_output/', userFilenames{3},
    ↪ '.png']);
    % Apply Canny edge detection to the cleaned image
    edgeImage = edge(cleanedImage, 'Canny');
    imwrite(edgeImage, ['img_output/', userFilenames{4}, '.png']);
end
```

Partial code of main_task.m

```
% Q5: Determine the outline(s) of characters in the image.
fprintf('Determining the outlines of characters in the
    ↪ image...\n');
edgeImage = getOutline(binaryImage, 'erodedImage', 'dilatedImage',
    ↪ 'cleanedImage', 'edgeImage');
```

This function aids in revealing distinct region outlines in binary images, making it useful for subsequent tasks. Now we can use the following code segment to identify character outlines in the image using the *getOutline* function.



Figure 8: The results of applying the *getOutline* function

2.6 Question 6

Determine the outline(s) of characters in the image

segmentAndLabelCharacters function has been defined to implement the segmentation and labeling of characters. The main steps include:

1. **Connected Component Analysis:** Utilizing the 'regionprops' function to conduct connected component analysis on the input binary image, extracting bounding box information for characters.
2. **Bounding Box Width Segmentation:** Iterating through the bounding boxes of each character, checking if the width exceeds the set threshold. For characters with widths surpassing the threshold, the bounding box is horizontally split into two parts to accommodate larger characters.
3. **Character Cropping and Saving:** Based on the segmented bounding box information, each character is cropped and saved to the specified output directory. Simultaneously, a new image is generated containing the visual representation of character bounding boxes.
4. **Displaying Bounding Boxes:** Displaying the bounding boxes of characters on the original binary image, using red rectangles to identify the position of each character. This step facilitates the intuitive representation of accurate character segmentation.

In summary, the function effectively achieves character segmentation and labeling through connected component analysis, bounding box width determination, character cropping and saving, as well as the visualization of bounding boxes. These functionalities not only form the foundation for character recognition but also provide a visual representation for further analysis and understanding. The specific code is provided below.

segmentAndLabelCharacters.m

```
function segmentAndLabelCharacters(binaryImage, outputDirectory)
    % Perform connected component analysis
    stats = regionprops(binaryImage, 'BoundingBox');
    % Set thresholds to determine if the bounding box width is
    ↪ greater than 150
    minWidth = 30;
    maxWidth = 150;
    % Create a new cell array to store all bounding box
    ↪ information
    newBoundingBox = cell(1, numel(stats));
    % Initialize index
    newIndex = 10;
    % Iterate through each bounding box
    for i = 1:numel(stats)
        boundingBox = round(stats(i).BoundingBox);
        % Check if the bounding box width is greater than the
        ↪ threshold
        if boundingBox(3) > maxWidth
            % Reduce the bounding box width to half (split into
            ↪ left and right)
            newWidth = round(boundingBox(3) / 2);
            % Store new bounding box information (left part)
            newBoundingBox{newIndex} = [boundingBox(1),
            ↪ boundingBox(2), newWidth, boundingBox(4)];
            newIndex = newIndex + 1;
            % Store new bounding box information (right part)
            newBoundingBox{newIndex} = [boundingBox(1) + newWidth,
            ↪ boundingBox(2), newWidth, boundingBox(4)];
            newIndex = newIndex + 1;
        else
            % Bounding box width is not greater than the
            ↪ threshold, store original bounding box information
            newBoundingBox{newIndex} = boundingBox;
            newIndex = newIndex + 1;
        end
    end
    % Initialize index
    newIndex = 1;
    ... (Due to space limitations, the code snippet is continued below)
```


segmentAndLabelCharacters.m

```

... (The code is continued from the previous snippet)
% Display bounding boxes on the original image
figure;
imshow(binaryImage);
hold on;
% Check if newBoundingBox is not empty
if ~isempty(newBoundingBox)
    % Iterate through newBoundingBox for uniform cropping and
    ↪ saving
        for i = 1:numel(newBoundingBox)
            boundingBox = round(newBoundingBox{i});
            boundingBox = boundingBox(:)';

            % Check if the bounding box size is greater than or
            ↪ equal to minWidth
            if boundingBox(3) >= minWidth
                % Crop the image
                croppedImage = imcrop(binaryImage, boundingBox);

                % Generate the filename for the cropped image
                outputFileName = sprintf('character%d.png',
                ↪ newIndex);
                newIndex = newIndex + 1;

                % Save the cropped image
                imwrite(croppedImage, fullfile(outputDirectory,
                ↪ outputFileName));

                % Display the bounding box
                x = boundingBox(1);
                y = boundingBox(2);
                width = boundingBox(3);
                height = boundingBox(4);
                rectangle('Position', [x, y, width, height],
                ↪ 'EdgeColor', 'r', 'LineWidth', 1);
            end
        end
    else
        % Handle the case when newBoundingBox is empty, e.g.,
        ↪ print a message
        disp('No bounding boxes to display.');
```

Partial code of main_task.m

```
% Q6: Segment the image to separate and label the different
↳ characters.
fprintf('Segmenting the image to separate and label the different
↳ characters...\n');
cleanedImage = imread('img_output\cleanedImage.png');
segmentAndLabelCharacters(cleanedImage,
↳ 'img_output\CroppedCharacters');
```



Figure 9: Separate and label the different characters

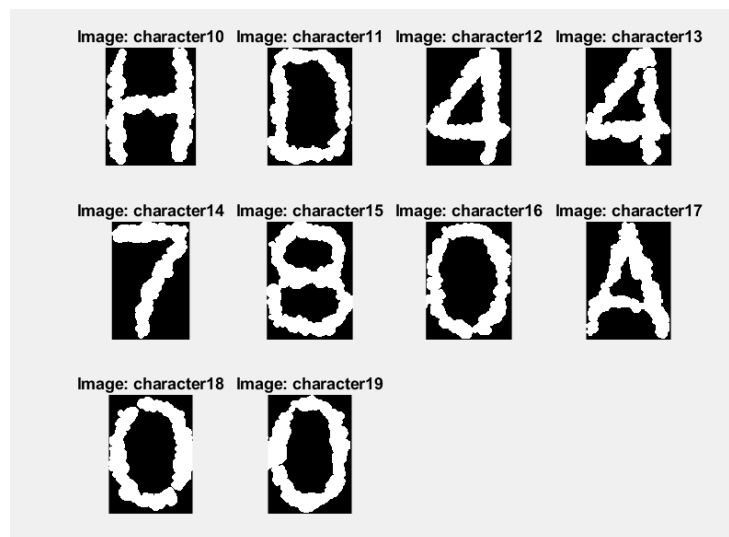


Figure 10: Segmented character images

2.7 Question 7

A dataset contained in the file `p_dataset_26.zip` is provided on CANVAS. Divide this dataset into two portions: (a) a 75% portion that will be used as the training set, and (b) the remaining 25% portion as the validation (testing) set. Use these two sub-datasets to complete the following three sub-tasks:

- **Sub-task 1:** Design a CNN to classify each character in the second line in Image 1, i.e., HD44780A00.
- **Sub-task 2:** Design a classification system, using a non-CNN-based method (or a combination of such methods) selected from those methods that have been covered in Part 2 of this course, to classify each character in the second line in Image 1, i.e., HD44780A00.
- **Sub-task 3:** Report the results obtained from Sub-task 1 and Sub-task 2, and compare the effectiveness and efficiency of the two approaches (i.e., one uses a CNN, the other does not). Provide your own explanation on any differences in the results between these two approaches.

2.7.1 Data Preparation and Division

The process of preparing and dividing the dataset can be broken down into the following steps:

1. **Setting Dataset Path:** This line establishes the variable path denoting the location of the dataset. The *fullfile* function is employed to construct a complete path from the specified folder name.
2. **Creating Image Datastore:** The code employs the *imageDatastore* function to instantiate an image datastore (*imds*) sourced from the dataset located at the designated path. The datastore is configured to encompass subfolders, and labels are derived from the folder names.
3. **Splitting Dataset into Training and Testing Sets:** Utilizing *splitEachLabel*, the dataset is partitioned into training (*imdsTrain*) and testing (*imdsTest*) sets. The split ratio is set at 75% for training and 25% for testing, employing a *randomized* approach where data points are randomly assigned to either training or testing sets.
4. **Determining the Number of Classes:** The code calculates the number of classes within the training set using the *categories* function. The result is stored in the variable *numClasses*.

2.7.2 Convolutional Neural Networks (CNN)

1. Neural Network Architecture To achieve the desired outcome, we have designed the following CNN structure:

- a) Input Layer: An image input layer designed for grayscale images with dimensions 128x128 pixels.
- b) Convolutional Blocks: Multiple convolutional blocks, each comprising a convolutional layer (*convolution2dLayer*), batch normalization layer (*batch-NormalizationLayer*), and Rectified Linear Unit (*ReLU*) activation function (*reluLayer*). These blocks are designed to capture hierarchical features in the input data.
- c) Pooling Layers: Max pooling layers (*maxPooling2dLayer*) are employed to reduce spatial dimensions, allowing the network to focus on the most salient features.
- d) Dropout Layer: A dropout layer (*dropoutLayer*) is incorporated to mitigate overfitting during the training process.
- e) Fully Connected Layer: A fully connected layer (*fullyConnectedLayer*) with seven output neurons, followed by a softmax activation layer (*softmaxLayer*) for multiclass classification.
- f) Output Layers: The architecture concludes with a classification layer (*classificationLayer*) to determine the class label.

2. Training Options The training process is configured using the 'trainOptions' object, specifying the following parameters:

- Optimization algorithm: Adam.
- Initial learning rate: 0.02.
- Maximum epochs: 22.
- Data shuffling: Conducted at the beginning of each epoch.
- Validation data: Utilizing the *imdsTest* dataset.
- Validation frequency: Every 20 mini-batches.
- Mini-batch size: 55.
- Verbosity: No command line output during training.
- Execution environment: Automatic selection between CPU and GPU.
- Plotting: Training progress plots for monitoring convergence.

3. Testing and Evaluation Post-training, the network undergoes evaluation on the test data (*imdsTest*) through the *classify* function. Subsequently, a confusion matrix is computed using *confusionmat*, and the results are visually represented using *plotconfusion*. The accuracy of the trained model is quantified by comparing predicted labels (*YPred*) with ground truth labels (*YTest*). The accuracy is computed as the ratio of correct predictions to the total number of predictions.

4. Storing Training Results After the training process, the code begins by specifying the directory where the trained model will be saved. It then checks whether this directory already exists. If the directory does not exist, the script creates it using the *mkdir* function. Next, the script specifies the name to be assigned to the saved model. The full path for saving the model is then created by combining the specified directory and name using the *fullfile* function. Finally, the trained CNN model (*net*) is saved to the specified full path using the *save* function. This code ensures that the trained model is stored in an organized manner within the specified directory.

2.7.3 Multilayer Perceptrons (MLP)

1. Neural Network Architecture

- a) Input Layer: An image input layer designed for grayscale images with dimensions 128x128 pixels.
- b) *fullyConnectedLayer(256)* This is the first fully connected layer with 256 neurons. The purpose of this layer is to map the features of the input data into a space with 256 nodes, where the batch normalisation layer using the function (*batchNormalizationLayer*) is used to speed up the training process and increase the stability of the network. This layer uses *reluLayer* which is an activation function that uses Rectified Linear Unit (*ReLU*) activation function. It introduces non-linear properties that allow the network to learn complex patterns.
- c) *fullyConnectedLayer(128)* This is the second fully connected layer with 128 neurons. The purpose of this layer is to map the features of the input data into a space with 128 nodes, it also consists of batch normalization layer (*batchNormalizationLayer*), and Rectified Linear Unit (*ReLU*) activation function (*reluLayer*).
- d) *fullyConnectedLayer(64)* This is the second fully connected layer with 64 neurons. The purpose of this layer is to map the features of the input data into a space with 64 nodes, it also consists of batch normalization layer (*batchNormalizationLayer*), and Rectified Linear Unit (*ReLU*) activation function (*reluLayer*).
- e) *fullyConnectedLayer(32)* This is the first fully connected layer with 32 neurons. The purpose of this layer is to map the features of the input data into

a space with 32 nodes, it consists of batch normalization layer (*batchNormalizationLayer*), and sigmoid activation function (*sigmoidLayer*), which is an activation function that restricts the output to between 0 and 1. Typically used for binary classification problems.

- f) *fullyConnectedLayer(7)* The last fully connected layer with 7 neurons corresponding to the number of categories of the network output, followed by a softmax activation layer (*softmaxLayer*) for multiclass classification.
- g) Output Layers: The architecture concludes with a classification layer (*classificationLayer*) to determine the class label.

2. Training Options The training process is configured using the *trainingOptions* object, specifying the following parameters:

- Optimization algorithm: sgdm.
- Initial learning rate: 0.008.
- Maximum epochs: 49.
- Mini-batch size: 49.

Other parameters and subsequent steps are the same as for CNN and are omitted here.

2.7.4 Results

1. Loading and Classification

- The script begins by specifying the directories for the pre-trained neural network model (*TrainedCNN* or *TrainedMLP*), the specific model's name, and the directory containing the cropped characters. The script then loads the pre-trained neural network model and retrieves all PNG files from the directory.
- Next, it initializes an array of structures to store the results of image classification. The script iterates over each test image, reads the image, and uses the pre-trained model to classify it. The results, including the filename, class label, and classification scores, are stored in the results structure array.
- Finally, a new figure is created to display the results. The script iterates over the stored results, displaying each image along with its classification results, including the assigned class label and the corresponding accuracy score. (*The accuracy is defined as the similarity between the classified and the ones in trained model*). The subplot arrangement is organized in two rows and a number of columns based on the number of test images.

2. CNN results

- The accuracy of the validation: 97.73%.
- The time for training: 52 seconds.
- The classified results of each character for CNN training are as shown in following Figure



Figure 11: The classified results of CNN training

It is evident that each character is classified correctly and the accuracy indicates that the pre-processed image matches the training model well. The accuracy of the seventh character, 55.71%, is the lowest value and the ones of both the fourth and sixth are lower than 80%.

3. MLP results

- The accuracy of the validation: 97.73%.
- The time for training: 52seconds.
- The classified results of each character for MLP training are as shown in Figure 12



Figure 12: The classified results of MLP training

The character of D and 8 are classified to wrong items and the incorrect similarity of this two are relatively high, even the percentage of 8 to be 93.6%, which might be an entire misclassification. In addition, the accuracy of those correct classification is relatively low and the value fluctuates between 50% and 80%.

Table 1: Comparison of CNN and MLP performances

	H	D	4	4	7	8	0	A	0	0
CNN	99.9	97.9	97.8	68.1	100	77	55.7	100	85.4	98.4
MLP	98.8	N/A	52.9	74.3	97.8	N/A	73.7	91.6	84.9	83.5

2.7.5 Comparison

1. Effectiveness The classification outcomes of CNN demonstrate a complete accuracy but two of characters are mis-classified by MLP training. Additionally, the accuracy attained by CNN exceeds that of MLP. Consequently, it is deduced that, considering the near entirety of the training data, CNN exhibits superior effectiveness compared to MLP.

CNN is highly effective for tasks involving spatial relationships and hierarchical patterns, especially in image-related tasks. CNN uses convolutional layers that can automatically learn hierarchical features from the input data, it excels at tasks like image classification, object detection, and image segmentation.

MLP is effective for tasks where the input data doesn't have a clear grid-like

structure, and there's no inherent notion of spatial relationships between data points. MLP processes each input feature independently, making it suitable for tasks where the relationships between features are not spatially dependent.

2. Efficiency Despite the high effectiveness of CNN, it is noteworthy that the time required for training is several-fold greater than that of MLP. The training process for CNNs is protracted and entails a heightened computational cost, whereas MLPs exhibit the capability to undergo training across numerous iterations within a more constrained temporal framework.

CNN is efficient in terms of parameter sharing due to weight sharing in convolutional layers. This allows them to learn and generalize patterns efficiently, especially in scenarios where translation-invariant features are important. Training a CNN can be computationally expensive, particularly for large and deep networks. However, the use of techniques like transfer learning (*pre-training on a large dataset and fine-tuning on a specific task*) can improve efficiency.

Training an MLP is generally computationally less expensive compared to training a CNN. The absence of convolutional and pooling layers reduces the overall model complexity. MLP may struggle with tasks that involve spatial dependencies or hierarchical patterns, as they lack the specialized layers (like convolutional layers) designed to capture such relationships.

2.8 Question 8

In carrying out Step 7, also experiment with pre-processing of the data (e.g., padding/resizing the input images) as well as with hyperparameter tuning. In your report, discuss your findings and how sensitive your approach is to these changes.

2.8.1 Pre-processing of Data

Throughout the training of neural networks, the implementation of preprocessing is imperative for optimizing the model's performance. A primary objective of preprocessing is to standardize or normalize input data, ensuring uniform feature scales and mitigating the disproportionate impact of specific features on model weights. Furthermore, preprocessing serves to eliminate noise and outliers from the data, thereby enhancing the model's robustness to better capture authentic data patterns.

The integration of data augmentation techniques, such as rotation, flipping, and scaling, during preprocessing contributes to broadening the diversity of the training set. This augmentation strategy enhances the model's generalization capability. Moreover, preprocessing aids in reducing the dimensionality of input data or constraining feature ranges, thereby diminishing computational complexity and expediting the model training process. Addressing missing values is also a vital aspect of preprocessing, guaranteeing the integrity of input

data.

In summary, meticulous selection and application of suitable preprocessing methods play a pivotal role in enhancing the efficiency, performance, and generalization of models. This, in turn, renders them more reliable in tackling intricate data and large-scale tasks. In this specific test, the training and testing sets will undergo preprocessing incorporating random rotation, random translation and random scaling. Additionally, a separate examination of each augmentation technique will be conducted to assess its impact and sensitivity in the given context.

1.All enhanced: Rotation, translation and scaling The result of CNN with all enhanced preprocessing is presented in *Figure 13*.

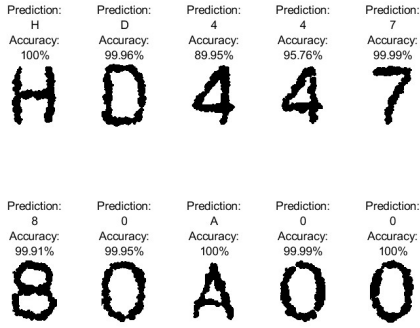


Figure 13: The accuracy of CNN and CNN with enhanced inputs

Table 2: The accuracy of MLP and MLP with enhanced inputs

	H	D	4	4	7	8	0	A	0	0
CNN	99.9	97.9	97.8	68.1	100	77	55.7	100	85.4	98.4
CNN_ALL	100	99.9	89.9	95.8	99.9	99.9	99.9	100	99.9	100

Following the implementation of rotation, translation, and scaling enhancements to the training set, the accuracy of each individual character approaches nearly 100%. Particularly noteworthy is the substantial improvement in accuracy for characters that initially exhibited low recognition rates. For instance, the accuracy of the seventh character '0' has surged from 55.7% to an impressive 99.9%. This outcome underscores the efficacy of padding and resizing the input data, revealing a pronounced enhancement in the training outcomes for the Convolutional Neural Network.

The result of MLP with all enhanced preprocessing is presented in figure 14.



Figure 14: The classified results of MLP with all enhanced inputs

Table 3: The accuracy comparison of CNN with rotation enhanced

	H	D	4	4	7	8	0	A	0	0
CNN	98.8	N/A	52.9	74.3	97.8	N/A	73.7	91.6	84.9	83.5
CNN_ALL	98.1	96.3	98.9	99.1	99.5	99.2	97.3	89.8	98.3	99.4

After preprocessing the training data with random rotation, translation, and scaling, the accuracy of the MLP on the validation set improved. The model can now classify each character with high accuracy, including previously challenging cases like the *D* and *8*, which can now be predicted with an accuracy of 96.3% and 99.2% respectively. This indicates that preprocessing the training data is beneficial for enhancing the neural network's generalization ability, increasing adaptability to various image variations, and ultimately improving model performance.

2.Only rotating enhanced preprocessing Image rotation processing stands as a potent data augmentation technique extensively employed in deep learning endeavors. Through the rotation and transformation of images within the training set, this technique affords the model a more diverse array of data, facilitating the acquisition of object features from distinct angles. This methodology not only enhances the model's capacity to accommodate rotational variations in input data but also bolsters its robustness, thereby mitigating the risk of overfitting and augmenting the model's generalization capability.

In visual tasks, particularly those necessitating the handling of objects from varied angles, image rotation processing emerges as a pivotal strategy to enhance model performance. The incorporation of this data augmentation technique enables more effective addressing of diverse rotation transformations encountered in practical applications, thereby furnishing robust support for the training and performance of deep learning models.

The result of CNN with rotating enhanced preprocessing is presented in following table.

Table 4: The accuracy comparison of CNN with rotation enhanced

	H	D	4	4	7	8	0	A	0	0
CNN	99.9	97.9	97.8	68.1	100	77	55.7	100	85.4	98.4
CNN_ALL	100	99.9	89.9	95.8	99.9	99.9	99.9	100	99.9	100
CNN_Rot	99.9	98.5	86.7	98.2	99.6	99.2	N/A	100	N/A	99.6

Following the rotational enhancement of the training set, there is a noticeable improvement in overall accuracy; however, a consequential rise in classification errors is observed, particularly in the identification of the character '0'. Notably, both the 7th and 9th characters are erroneously classified as 'D'. This occurrence is likely attributed to the fact that, post-rotation, the Convolutional Neural Network faces challenges in distinctly discerning between the features of '0' and 'D'. This underscores the necessity for additional enhancements to supplement the training process, particularly for features with closely aligned outer profiles.

The result of MLP with rotating enhanced preprocessing is presented in following table.

Table 5: The accuracy comparison of MLP with rotation enhanced

	H	D	4	4	7	8	0	A	0	0
MLP	98.8	N/A	52.9	74.3	97.8	N/A	73.7	91.6	84.9	83.5
MLP_ALL	98.1	96.3	98.9	99.1	99.5	99.2	97.3	89.8	98.3	99.4
MLP_Rot	98.7	N/A	71.6	85.7	98.9	N/A	67.5	97.6	92.0	94.5

After applying rotation preprocessing to the experimental data and keeping the original hyperparameters unchanged, the accuracy of the MLP on the validation set improved, but the prediction accuracy decreased. Two data points were not predicted correctly, and despite multiple tests and hyperparameter adjustments, the situation persisted. The analysis suggests that the decline in prediction accuracy when rotating the data may be attributed to information loss and excessive deformation. Therefore, performing rotation preprocessing alone appears to reduce the predictive accuracy of this neural network.

3.Only translation enhanced preprocessing The implementation of translation preprocessing on experimental data serves to bolster the robustness and generalization capability of neural networks. This technique facilitates the model in acquiring features of objects across diverse positions within images, thereby diminishing sensitivity to alterations in object position and enhancing accurate object recognition. The infusion of random translations during training results in the generation of varied samples, thereby enhancing the model's adaptability to different positions and mitigating the risk of overfitting. In essence,

translation preprocessing enables the neural network to holistically grasp object features, augmenting its comprehension of the entire data distribution and consequently improving generalization.

The result of CNN with translation enhanced preprocessing is presented as follows.

Table 6: The accuracy comparison of CNN with translation enhanced

	H	D	4	4	7	8	0	A	0	0
CNN	99.9	97.9	97.8	68.1	100	77	55.7	100	85.4	98.4
CNN _ALL	100	99.9	89.9	95.8	99.9	99.9	99.9	100	99.9	100
CNN _Tra	99.7	98.5	84.5	N/A	99.9	93.8	N/A	100	86.2	97.3

Following the application of translation enhancement, characters that were initially recognized correctly exhibit negligible change in accuracy. However, less accurate characters, such as '4' and '0', experience misclassifications, with '4' being erroneously identified as 'A' and '0' as 'D'. This misclassification is likely attributed to the displacement of features during the translation operation, introducing confusion during model training. The model appears to be trained in a manner inconsistent with the intended categorization of features, emphasizing the importance of aligning features with their respective categories. Consequently, it is apparent that translation enhancement alone heightens the probability of misclassification.

The result of MLP with translation enhanced preprocessing is presented as follows.

Table 7: The accuracy comparison of MLP with translation enhanced

	H	D	4	4	7	8	0	A	0	0
MLP	98.8	N/A	52.9	74.3	97.8	N/A	73.7	91.6	84.9	83.5
MLP _ALL	98.1	96.3	98.9	99.1	99.5	99.2	97.3	89.8	98.3	99.4
MLP _Tra	98.5	96.7	64.9	82.5	98.9	56.6	76.9	95.7	N/A	96.9

After applying translation preprocessing to the experimental data and keeping the original hyperparameters unchanged, the accuracy of the MLP on the validation set improved. However, the prediction accuracy did not increase, even after multiple adjustments to the hyperparameters. All letters could not be correctly predicted. Through analysis, it is suggested that the reason for the decline in accuracy after translation preprocessing may be attributed to information loss, positional shifts, and insufficient model complexity.

4.Only scaling enhanced preprocessing In neural network learning, scaling preprocessing of experimental data offers the advantage of enhancing the model's adaptability to objects and scenes at different scales. By introducing scale variations, the model can comprehensively learn object features, thereby

increasing its generalization capability across various scaling levels. This operation also helps alleviate model overfitting to specific scales, enhancing its robustness in real-world applications. In summary, scaling preprocessing contributes to making the neural network more resilient, adapting to image variations at different scales. The result of CNN with scaling enhanced preprocessing is shown in following table.

Table 8: The accuracy comparison of CNN with scaling enhanced

	H	D	4	4	7	8	0	A	0	0
CNN	99.9	97.9	97.8	68.1	100	77	55.7	100	85.4	98.4
CNN _ALL	100	99.9	89.9	95.8	99.9	99.9	99.9	100	99.9	100
CNN _Sca	99.7	100	99.4	N/A	100	99.2	N/A	100	72.6	97.2

Upon augmenting the training set with scaling, there is a lack of significant change in overall accuracy. However, characters that were initially prone to accurate identification, such as 4 and 0, undergo misclassification. Specifically, 4 is now identified as 0, and 0 is identified as D. This suggests that scaling alone introduces additional features to the original model, leading to confusion, and thus, solely relying on scaling not only fails to contribute to accuracy improvement but also significantly increases the likelihood of misclassification.

The result of MLP with scaling enhanced preprocessing is shown in following table.

Table 9: The accuracy comparison of MLP with scaling enhanced

	H	D	4	4	7	8	0	A	0	0
MLP	98.8	N/A	52.9	74.3	97.8	N/A	73.7	91.6	84.9	83.5
MLP _ALL	98.1	96.3	98.9	99.1	99.5	99.2	97.3	89.8	98.3	99.4
MLP _Sca	98.4	69.1	N/A	95.8	98.7	86.7	94.4	96.6	98.4	99.4

After applying scaling preprocessing to the experimental data and keeping the original hyperparameters unchanged, the accuracy of the MLP on the validation set improved. However, the prediction accuracy did not increase. During the process of adjusting hyperparameters, it was observed that increasing the value of *MaxEpochs* and thus the number of iterations resulted in improved prediction accuracy. This adjustment allowed the model to accurately predict all letters, as shown in Figure 9. The decrease in accuracy may be attributed to the emergence of new features and changes in the data after scaling, requiring more training time for the model to adapt to these new variations. Therefore, increasing the value of *MaxEpochs* provides the model with more opportunities to learn and adjust weights, allowing it to better adapt to the changes introduced by scaling.

5. Conclusion

- The implementation of rotation, translation, and scaling enhancements in the training set significantly improves the accuracy of individual characters, particularly benefiting those with initially low recognition rates. However, post-rotation, there is an overall accuracy improvement coupled with increased classification errors, emphasizing the need for supplementary enhancements to address challenges in distinguishing features. Translation enhancement leads to negligible changes in accuracy for correctly recognized characters but introduces misclassifications, underscoring the importance of aligning features with their intended categories. Scaling, when added to the training set, fails to bring about significant overall accuracy changes but increases the likelihood of misclassification, indicating the necessity for a comprehensive approach to feature enhancement in Convolutional Neural Network training.
- Applying rotation, translation, and scaling data augmentation techniques to experimental data is beneficial for increasing the diversity of the training set and improving the model's generalization capability. In Multilayer Perceptron experiments, using each of these three methods individually resulted in an improvement in the accuracy of the validation set. However, the prediction outcomes varied when each method was applied separately: using random rotation and translation alone led to a decrease in prediction accuracy, while using scaling alone with an increased number of iterations improved prediction accuracy. Combining all three data processing methods achieved accuracy of 95% or higher for all letters. In summary, when preprocessing data, adopting multiple data augmentation techniques and adjusting the number of iterations can enhance the model's adaptability to the diverse characteristics of updated data, leading to more accurate predictions.

2.8.2 Hyperparameter Tuning

The hyperparameters are crucial for the training process and can significantly impact the performance of the neural network. Fine-tuning these hyperparameters is often necessary to achieve optimal results for a specific problem and dataset. Adjusting hyperparameters involves experimentation and iterative refinement based on the observed model performance during training and validation. The hyperparameter tuning of both CNN and MLP focuses on the following three indicators: Initial Learning Rate, Maximum Epochs and Minimum Batch Size.

1. **Parameter Ranges and Initialization** The section defines a range for the maximum number of training parameter (Initial Learning Rate, Maximum Epochs and Minimum Batch Size) and generates 10 evenly spaced values within this range. It also initializes an empty structure array to store the results of the hyperparameter tuning.
2. **Hyperparameter Tuning Loop** The loop iterates over the different

values of hyperparameter, updates the options structure, trains a neural network (net), evaluates its performance on a test set (*imsTest*), calculates accuracy, and stores the results in the results structure.

3. **Displaying Results** The section displays the results of the hyperparameter tuning, showing the learn rate, max epochs, mini-batch size, and accuracy for each set of hyperparameters.
4. **Extracting and Analyzing Results** The max parameters and accuracy results are extracted from the results structure. It then identifies the hyperparameters that resulted in the highest accuracy to be the optimal values.

1. Initial Learning Rate *InitialLearnRate*, the initial learning rate for the Adam optimizer, is a hyperparameter that determines the step size during the operation process. The results of CNN and MLP are shown as below.

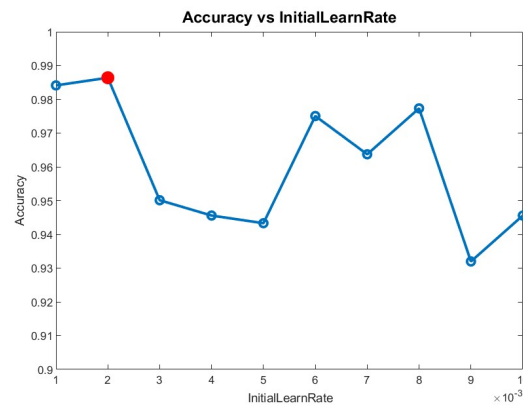


Figure 15: The accuracy of CNN with changing Initial Learning Rate

There is no clear linear relationship between accuracy and Learning Rate, with accuracy values fluctuating from 93% to 99%. The maximum of accuracy occurs on the value of Learning Rate to be 0.02, which is taken as the most optimal parameter.

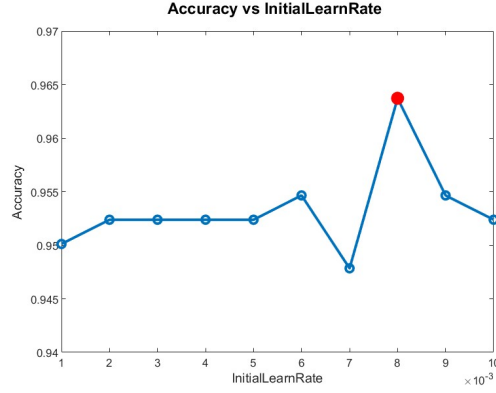


Figure 16: The accuracy of MLP with changing Initial Learning Rate

The Stochastic Gradient Descent with Momentum (*Sgdm*) optimizer is employed in the Multilayer Perceptron (MLP). The momentum mechanism introduced by Sgdm serves to expedite the convergence process, mitigate gradient oscillations, effectively navigate local minima, and enhance the model's generalization ability. The hyperparameter *InitialLearnRate* represents the initial learning rate of the Sgdm optimizer, determining the step size during execution. Notably, there is no discernible linear relationship between accuracy and the learning rate, as accuracy values fluctuate within the range of 94% to 97%. The peak accuracy is observed at a learning rate of 0.08, identified as the most optimal parameter for the model.

2. Minimum Batch Size This hyperparameter defines the maximum number of training epochs. It sets an upper limit on the number of passes through the entire training dataset. The results are presented as follows.

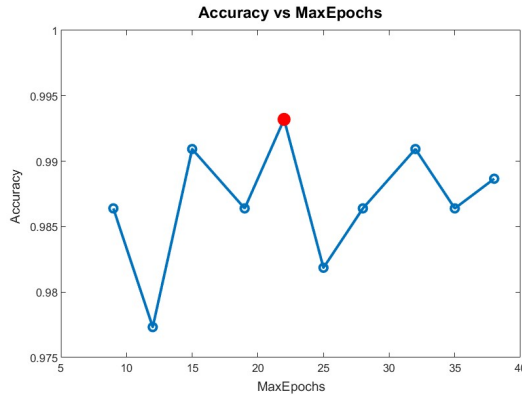


Figure 17: The accuracy of CNN with changing Maximum Epochs

Maximum Epochs is a non-linear relationship for accuracy and the value of accuracy is less affected by the change in alteration parameter and only fluctuates from 98% to 99.5%. The most significant effect of adjusting this parameter lies in the training time of the CNN, the larger the value of this parameter is, the more iterations there are, and the longer it takes to train the model.

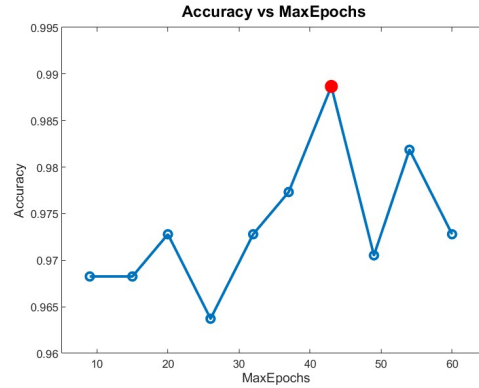


Figure 18: The accuracy of MLP with changing Maximum Epochs

The relationship between the Maximum Epochs and accuracy is non-linear, with the accuracy value demonstrating minimal sensitivity to alterations in the epoch parameter, fluctuating within the range of 96% to 99%. Excessively high values of Maximum Epochs can induce overfitting, characterized by optimal performance on the training set but suboptimal performance on new data. Conversely, setting the parameter too low may hinder the model's ability to fully grasp the underlying data features, thereby impacting overall performance. Therefore, meticulous adjustment of the Maximum Epochs parameter is essential to strike a balance between effectively learning data patterns and preventing overfitting on the training set.

3. Minimum Batch Size The mini-batch size is a hyperparameter that determines the number of training examples used in each iteration of the optimization algorithm. The differences between CNN and MLP are demonstrated as follows.

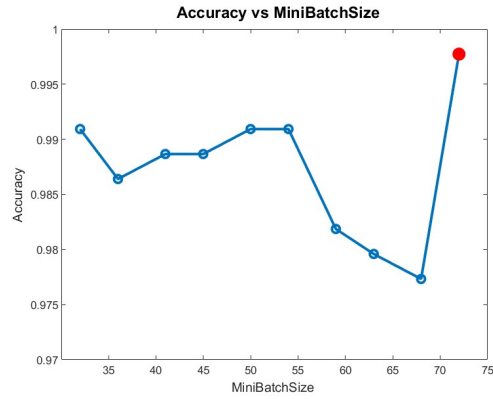


Figure 19: The accuracy of CNN with changing Minimum Batch Size

Minimum Batch Size has a small effect on accuracy, with accuracy around 99% until the value reaches 60. However, as the value increases, there is a sudden change in accuracy and the highest accuracy is produced at the largest value tested, which could be due to chance during training. Based on this result, the parameter corresponding to the highest accuracy in the graph should not be chosen, but rather in a more stable interval of accuracy, for example 55 is taken as the parameter for CNN.

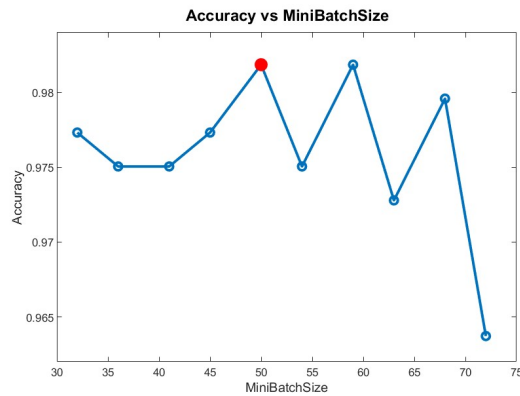


Figure 20: The accuracy of MLP with changing Minimum Batch Size

The Minimum Batch Size demonstrates a continuous increment until reaching 49, where it achieves its maximum accuracy of 98.3%. However, beyond a Minimum Batch Size of 49, accuracy begins to fluctuate. Therefore, 49 is considered the optimal choice for the Minimum Batch Size in the context of our performance testing.

4. Conclusion

- In conclusion, the optimization of key parameters in the Convolutional Neural Network (CNN) plays a pivotal role in enhancing its performance. The *InitialLearnRate* hyperparameter, determining the initial learning rate for the Adam optimizer, showcases a non-linear relationship with accuracy, where the optimal parameter is identified at a learning rate of 0.02. Meanwhile, the exploration of Maximum Epochs reveals its non-linear correlation with accuracy, with adjustments primarily influencing the training time of the CNN rather than significantly impacting accuracy values. Lastly, the Minimum Batch Size demonstrates a modest effect on accuracy, with a recommended parameter of 55 selected from a stable interval, avoiding potential chance effects associated with extreme values. These findings underscore the importance of meticulous parameter tuning to achieve optimal CNN performance, balancing accuracy and training efficiency.
- In summary, incorporating the Stochastic Gradient Descent with Momentum (*Sgdm*) optimizer in the Multilayer Perceptron (MLP) proves highly advantageous for efficient model convergence, gradient stability, and enhanced generalization. The *InitialLearnRate* hyperparameter, optimized at 0.08, demonstrates peak accuracy, emphasizing the importance of meticulous parameter tuning. The non-linear relationship between Maximum Epochs and accuracy highlights the need for careful parameter adjustment to prevent overfitting, ensuring a delicate balance in learning data patterns. Additionally, the exploration of Minimum Batch Size identifies 49 as the optimal choice, achieving maximum accuracy (98.3%) in performance testing. Overall, these findings underscore the critical role of parameter tuning in optimizing the MLP's performance, ensuring robust training and generalization within the study's scope.

3 Summary

The project stands as a sophisticated exploration at the confluence of image processing and machine learning, honing in on character classification within microchip labels. Early stages entail crucial preprocessing steps, including grayscale conversion and smoothing techniques, foundational for refining the dataset and readying it for advanced analyses.

A pivotal undertaking involves extracting specific lines from images and creating binary images through thresholding. The precision achieved in character outlining, employing morphological operations and Canny edge detection, lays the groundwork for robust character recognition. This meticulous process ensures accurate segmentation of individual characters within microchip labels.

Advanced techniques, such as connected component analysis and bounding box width segmentation, elevate the character labeling process, enhancing overall recognition. Two distinct training models, Convolutional Neural Networks (CNN) and Multilayer Perceptrons (MLP), play crucial roles. While CNNs demonstrate high accuracy, their extended training periods contrast with the computational efficiency of MLPs, rendering them valuable alternatives.

The testing phase integrates enhanced pre-processing and Hyperparameter Tuning, significantly benefiting both CNN and MLP models. Techniques like rotation, translation, and scaling enhancements contribute to improved character recognition accuracy. Nuanced adjustments during hyperparameter tuning aim to strike a delicate balance between achieving high accuracy and ensuring training efficiency.

Experimentation within specified parameter ranges remains a linchpin of the project, enabling fine-tuning of models and identification of optimal configurations for peak performance. The interdisciplinary nature of this project, merging image processing and machine learning, underscores its pivotal role in advancing computer vision and pattern recognition systems.
